

UNIVERSIDAD DEL VALLE DE GUATEMALA

Ingeniera de Software 1 - Sección 30

Lynette García



**Plataforma digital para la colaboración interdisciplinaria
entre asociaciones estudiantiles universitarias**

Angel Gabriel Sanabria Morales - 24725

Saul Esteban Castillo Arenas - 24915

Vernel Josué Hernández Cáceres - 24584

Derek Friedhelm Coronado Chilin - 24732

Samuel Antonio Robledo López - 241282

Guatemala, 2026

Resumen

Sprint 3 del proyecto UVG Collab, una plataforma digital orientada a facilitar la colaboración interdisciplinaria entre asociaciones estudiantiles universitarias. Esta iteración se desarrolló entre el 23 de abril y el 11 de mayo de 2026, con el propósito de consolidar la plataforma desde dos áreas principales: la preparación del sistema para producción y la incorporación de nuevas funcionalidades para estudiantes y líderes.

Durante este sprint se trabajaron diez bloques principales del backlog: hosteo en servidor Azure con deploy automatizado, tareas transversales del sprint, edición de proyectos existentes, notificación al líder por nueva postulación, notificaciones generales del sistema, visualización del equipo actual del proyecto, cancelación de postulaciones enviadas, autocompletado del correo institucional, proyectos destacados y recuperación de contraseña. Estas funcionalidades complementan los avances de los sprints anteriores, donde ya se habían construido flujos base como autenticación, exploración, postulación, gestión de perfil, creación de proyectos, búsqueda avanzada y notificaciones iniciales.

El Sprint 3 también incluyó actividades relevantes de infraestructura y validación, como la configuración del entorno productivo en Azure, el uso de Nginx como reverse proxy, la definición del prefijo global /api, la revisión del pipeline de GitHub Actions, la documentación de variables y puertos, pruebas de integración, corrección de errores, revisión de seguridad y preparación de la presentación. Con esto, el equipo avanzó hacia una versión más madura del sistema, tanto desde el punto de vista funcional como desde la perspectiva de despliegue.

Los resultados del sprint fueron positivos, ya que todas las tareas planificadas fueron cerradas en estado completado. La métrica de velocidad muestra que el trabajo completado coincidió con el trabajo comprometido, mientras que el burndown chart confirma que los puntos pendientes llegaron a cero al cierre del sprint. Aun así, el gráfico también muestra que el avance no fue completamente uniforme, por lo que se mantiene como oportunidad de mejora cerrar tareas de forma más incremental y fortalecer la validación continua del sistema.

Introducción

El presente informe documenta el desarrollo y los resultados obtenidos durante el Sprint 3 del proyecto UVG Collab, una plataforma digital para facilitar la colaboración interdisciplinaria entre asociaciones estudiantiles universitarias. Después de los avances de los sprints anteriores, donde se consolidaron funcionalidades como autenticación, exploración de proyectos, postulación, gestión de perfil, creación de proyectos, búsqueda avanzada y notificaciones iniciales, esta tercera iteración se enfocó en fortalecer el sistema desde dos áreas principales: el despliegue en producción y la ampliación de funcionalidades clave para líderes y estudiantes.

Durante este sprint, el equipo trabajó en la configuración del hosteo en un servidor Azure, la integración de Nginx como reverse proxy, la validación del pipeline de despliegue automatizado y la documentación del entorno productivo. Estas tareas permitieron preparar la plataforma para dejar de depender únicamente del entorno local y acercarla a un escenario real de uso, accesible mediante una configuración de producción.

Además, se incorporaron mejoras funcionales importantes, como la edición de proyectos existentes, la visualización del equipo actual de un proyecto, la cancelación de postulaciones pendientes, el autocompletado del correo institucional, la sección de proyectos destacados, la recuperación de contraseña y la extensión del sistema de notificaciones. Estas funcionalidades complementan los flujos ya existentes y hacen que la plataforma sea más completa, útil y cercana a las necesidades de sus usuarios.

El sprint también incluyó tareas transversales de integración, corrección de errores, revisión de seguridad, validación del deploy y preparación de la presentación. En conjunto, estas actividades buscaron asegurar que las nuevas funcionalidades estuvieran correctamente conectadas entre frontend, backend, base de datos e infraestructura, manteniendo la protección mediante JWT y la consistencia general del sistema.

Alcance del Sprint 3

El Sprint 3 representa una etapa de consolidación técnica y funcional dentro del desarrollo de UVG Collab. A diferencia de los sprints anteriores, centrados principalmente en construir y ampliar el flujo base de la plataforma, esta iteración tuvo como objetivo preparar el sistema para producción y completar funcionalidades relacionadas con la administración de proyectos, postulaciones, notificaciones y acceso de usuarios.

En esta iteración se abordaron las siguientes áreas:

- **Hosteo en servidor Azure con deploy automatizado (HU-62):** configuración del entorno productivo, uso de Nginx como reverse proxy, definición del prefijo global /api para el backend, validación de puertos, revisión del pipeline de GitHub Actions y documentación de Docker Compose, variables de entorno y configuración de red.
- **Ver equipo actual del proyecto (HU-28):** desarrollo de un endpoint protegido para consultar colaboradores aceptados y una vista en el dashboard que muestra nombre, carrera, habilidades y rol asignado.
- **Editar proyecto existente (HU-26):** implementación del flujo para que el líder pueda actualizar información de un proyecto ya creado mediante un endpoint protegido y una vista de edición con datos precargados.

- **Notificación al líder por nueva postulación (HU-22):** extensión del flujo de postulaciones para generar una notificación automática al líder cuando un estudiante se postula a uno de sus proyectos.
- **Notificaciones generales del sistema (HU-12):** ampliación del servicio y de la interfaz de notificaciones para cubrir eventos adicionales, como cambios de estado de proyectos y nuevas postulaciones.
- **Cancelar postulación enviada (HU-20):** creación del flujo para que un estudiante pueda cancelar una postulación pendiente, validando que sea el dueño de la solicitud y que esta aún no haya sido evaluada.
- **Autocompletar correo institucional (HU-61):** mejora del formulario de registro para sugerir automáticamente el correo @uvg.edu.gt a partir del carné ingresado.
- **Ver proyectos destacados (HU-59):** incorporación de un endpoint y una sección en el dashboard para mostrar proyectos publicados ordenados por popularidad o cantidad de postulaciones.
- **Recuperación de contraseña (HU-14):** implementación del flujo para solicitar recuperación por correo, validar un token temporal y actualizar la contraseña de forma segura.
- **Tareas transversales Sprint 3 (HT-03):** pruebas de integración, corrección de errores, ajustes visuales, revisión de seguridad, validación en producción y preparación de la presentación del sprint.

En general, el Sprint 3 permitió avanzar hacia una versión más madura de la plataforma. El equipo no solo añadió funcionalidades visibles para los usuarios, sino que también fortaleció aspectos técnicos necesarios para operar en un entorno productivo, mejorar la seguridad de los endpoints y preparar una entrega más completa para la revisión académica.

Objetivos del Sprint

Los siguientes objetivos fueron priorizados por el equipo como los resultados de negocio más importantes. Representan los cambios medibles en el comportamiento del cliente que indicarian que la solución está funcionando.

- **Objetivo Principal - Adopción y Crecimiento**
 - Mayor adopción de la plataforma por asociaciones estudiantiles.

- Que los líderes de asociaciones y estudiantes elijan usar la plataforma de forma activa y recurrente como su canal oficial para publicar, explorar y gestionar proyectos extracurriculares.
- Métricas de éxito sugeridas
 - Número de asociaciones registradas activamente en la plataforma.
 - Frecuencia de acceso semanal por usuario.
 - Porcentaje de proyectos publicitarios a través del canal oficial vs. Canales informales.
- **Objetivo Secundario – Participación y Engagement**
 - Aumento en la participación estudiantil en proyectos extracurriculares
 - Que más estudiantes encuentren, postulen y completen proyectos dentro de la plataforma, motivados por información clara, filtros de interés y seguimiento visible de su progreso.
 - Métricas de éxito sugeridas
 - Número de postulaciones completadas por mes.
 - Tasa de retención en proyectos.
 - Reducción en el abandono de proyectos por falta de información o seguimiento
 - Número de estudiantes activos con al menos 1 proyecto en seguimiento.
- **Objetivo Estratégico - Reconocimiento Institucional**
 - Mayor validación institucional del esfuerzo estudiantil.
 - Que la universidad valide y registre institucionalmente las horas de participación extracurricular de los estudiantes a través de la plataforma, generando constancias digitales verificables como incentivo de uso.
 - Métricas de éxito sugeridas
 - Cantidad de horas beca/extensión validadas institucionalmente a través de la plataforma.
 - Número de constancias digitales emitidas por semestre.
 - Porcentaje de estudiantes que reportan reconocimiento institucional como incentivo principal de uso.

LeanUX

Sección	Contenido
Problema del negocio	La plataforma no alcanza sus objetivos porque la colaboración sigue siendo informal y fragmentada, sin herramientas integradas ni canales oficiales
Usuarios y clientes	Estudiantes universitarios que buscan colaborar en proyectos extracurriculares de forma estructurada y con validación institucional.
Beneficio a usuarios	Encontrar oportunidades rápido, seguimiento continuo, validación institucional verificable

Ideas de solución	Buscador filtrado, panel de horas, notificaciones, constancia digital, formulario de publicación, tablero de seguimiento
Hipótesis clave	Si la plataforma centraliza y organiza oportunidades con seguimiento, los estudiantes adoptarán la plataforma y obtendrán mayor reconocimiento institucional
Qué aprender primero	Elaborar un prototipo básico navegable y probarlo con usuarios reales

Product Backlog

ID	Historia de Usuario	Descripción	Actor principal	Prioridad	Estado	Sprint	Notas (estado del código)
Historias Técnicas (HT)							
HT-01	Preparación presentación, guion y revisión (S1)	Elaborar presentación, guion, flujo y revisar errores para la entrega del Sprint 1.	Equipo	Alta	✓ Completada	Sprint 1	
HT-02	Tareas transversales Sprint 2	ESLint backend, mejora consistencia visual UI, pruebas manuales flujo completo y preparación entrega Sprint 2.	Equipo	Alta	✓ Completada	Sprint 2	<i>eslint.config.mjs ya existía en frontend desde Sprint 1.</i>
HT-03	Tareas transversales Sprint 3	Pruebas de integración, corrección de errores, verificación del deploy en Azure y preparación de entrega del Sprint 3.	Equipo	Alta	✓ Completada	Sprint 3	<i>Pruebas de integración del flujo completo, corrección de errores, revisión de seguridad, validación del deploy en Azure y preparación de presentación del Sprint 3.</i>
HT-04	Tareas transversales Sprint 4	Tareas transversales que surjan durante el Sprint 4.	Equipo	Alta	— No iniciada	—	
Infraestructura y Configuración							
HU-00	Configuración e infraestructura del proyecto	Configurar entorno de trabajo: monorepo, BD PostgreSQL, Docker, Prisma ORM, seed inicial con 18 enums y 30+ modelos.	Equipo	Alta	✓ Completada	Sprint 1	<i>Monorepo, Docker, Prisma, seed.ts completamente funcional.</i>
HU-62	Hosteo en servidor Azure con deploy automatizado	Como equipo necesitamos desplegar la aplicación en Azure con pipeline de deploy automatizado para que la plataforma sea accesible en producción.	Equipo	Alta	✓ Completada	Sprint 3	<i>Configuración de Nginx como reverse proxy, prefijo global /api en backend, pipeline CI/CD con GitHub Actions, Prisma en producción y documentación del entorno Azure.</i>
Autenticación y Perfil de Usuario							
HU-01	Registro e inicio de sesión	Como estudiante deseo registrarme e iniciar sesión para acceder a la plataforma.	Estudiante	Alta	✓ Completada	Sprint 1	<i>POST /auth/login, POST /auth/register, JWT, validación @uvg.edu.gt.</i>
HU-02	Completar perfil académico	Como estudiante deseo completar mi perfil con carrera, habilidades, intereses y cualidades.	Estudiante	Alta	✓ Completada	Sprint 2	<i>Endpoints GET/PATCH /usuarios/me/perfil y PUT habilidades/intereses/cualidades. Form completo con React Hook Form + Zod.</i>

HU-13	Editar perfil académico	Como estudiante deseo editar mi perfil para mantener actualizadas mis habilidades e intereses.	Estudiante	Alta	✓ Completada	Sprint 2	Reutiliza formulario HU-02 en modo edición con datos precargados desde bootstrap endpoint.
HU-14	Recuperación de contraseña	Como usuario deseo recuperar mi contraseña mediante correo electrónico en caso de olvidarla para no perder el acceso a mi cuenta.	Estudiante	Media	✓ Completada	Sprint 3	Endpoints POST /auth/forgot-password y POST /auth/reset-password implementados. Flujo con token temporal, validación de token, actualización de contraseña con bcrypt y UI de recuperación/restablecimiento.
HU-15	Subir y actualizar foto de perfil	Como estudiante deseo subir una foto de perfil para personalizar mi cuenta.	Estudiante	Baja	— Parcialmente terminada	—	updateFotoUrl() existe en users.service. Cloudinary configurado. Falta UI de subida.
HU-16	Cerrar sesión de forma segura	Como usuario deseo cerrar sesión para proteger mi cuenta en dispositivos compartidos.	Estudiante	Alta	✓ Completada	Sprint 1	Botón Logout en DashboardLayout.tsx elimina token y redirige a /login.
HU-61	Autocompletar correo con carné o apellido	Como usuario deseo que el sistema complete automáticamente mi correo institucional a partir de mi carné o apellido para agilizar el registro.	Estudiante	Media	✓ Completada	Sprint 3	Autocompletado del correo institucional @uvg.edu.gt en el formulario de registro usando estado controlado en React.

HU-05	Postularse a un rol en proyecto	Como estudiante deseo postularme a un rol específico dentro de un proyecto.	Estudiante	Alta	✓ Completada	Sprint 1	POST /postulaciones. Vista /postular/[rollId] con form Zod, validaciones frontend y backend.
HU-06	Consultar mis postulaciones	Como estudiante deseo ver el historial y estado de todas mis postulaciones.	Estudiante	Alta	✓ Completada	Sprint 1	GET /postulaciones/mis-postulaciones. Vista con badges de estado por postulación.
HU-20	Cancelar postulación enviada	Como estudiante deseo cancelar una postulación pendiente en caso de cambiar de opinión antes de que el líder la evalúe.	Estudiante	Media	✓ Completada	Sprint 3	Endpoint DELETE /postulaciones/:id implementado con validación de dueño y estado PENDIENTE. Botón de cancelar agregado en Mis Postulaciones con modal de confirmación y refresco de lista.
HU-21	Notificación de resultado de postulación	Como estudiante deseo ser notificado dentro de la plataforma cuando mi postulación sea aceptada o rechazada.	Estudiante	Media	✓ Completada	Sprint 2	Módulo NotificationsService completo. Endpoints GET/PATCH notificaciones. Campana en navbar.
Creación y Gestión de Proyectos (Líder)							
HU-07	Crear y publicar proyecto	Como líder de asociación deseo crear y publicar proyectos con roles y requisitos de habilidad definidos.	Líder	Alta	✓ Completada	Sprint 2	POST /proyectos extendido con CreateProjectFullDto. Formulario multi-paso 3 pasos. Transacción Prisma.
HU-22	Editar proyecto existente	Como líder deseo editar la información de un proyecto ya creado para corregir errores o actualizar detalles.	Líder	Media	✓ Completada	Sprint 3	Endpoint PATCH /proyectos/:id implementado con validación del creador. Vista de edición con datos precargados y reutilización del formulario de creación. En Jira Sprint 3 registrado como IESUC-138 / HU-26.
HU-23	Definir roles y requisitos del proyecto	Como líder deseo definir los roles disponibles junto con habilidades requeridas.	Líder	Media	— Parcialmente terminada	—	Modelos RolProyecto y RequisitoHabilidad en schema. Cubierto parcialmente por HU-07.
HU-24	Definir hitos y cronograma del proyecto	Como líder deseo establecer hitos con fechas estimadas de entrega dentro del proyecto.	Líder	Media	— Parcialmente terminada	—	Modelo Hito existe en schema. hitos-section.tsx solo lectura en frontend.
HU-25	Cambiar estado del proyecto	Como líder deseo cambiar el estado de mis proyectos siguiendo transiciones válidas (borrador → publicado...).	Líder	Media	✓ Completada	Sprint 2	PATCH /proyectos/:id/estado con tabla de transiciones. Vista Mis Proyectos con selector.
HU-26	Archivar o eliminar proyecto	Como líder deseo archivar o eliminar un proyecto inactivo para mantener limpia la plataforma.	Líder	Baja	— Parcialmente terminada	—	Campo eliminadoEn en modelo (soft delete). Sin endpoint ni UI.
Gestión de Postulaciones (Líder)							
HU-08	Gestionar postulaciones recibidas	Como líder deseo aceptar o rechazar postulaciones para formar el equipo de trabajo de mi proyecto.	Líder	Alta	✓ Completada	Sprint 2	GET /proyectos/:id/postulaciones. PATCH /postulaciones/:id/estado. Vista agrupada por rol.
HU-27	Ver perfil del postulante	Como líder deseo ver el perfil completo de un	Líder	Media	— No iniciada	—	

		postulante para evaluar su idoneidad.					
HU-28	Enviar mensaje al postulante	Como líder deseo enviar un mensaje directo al postulante durante el proceso de evaluación.	Líder	Baja	— No iniciada	—	
HU-28	Ver equipo actual del proyecto	Como líder deseo ver la lista de colaboradores aceptados en mi proyecto con sus perfiles para conocer al equipo que formé.	Líder	Alta	✓ Completada	Sprint 3	Endpoint GET /proyectos/:id/equipo y vista /dashboard/proyectos/:id/equipo con tarjetas de colaboradores aceptados, carrera, habilidades y rol asignado. En Jira Sprint 3 registrado como IESUC-135 / HU-28.
Gestión de Tareas y Seguimiento							
HU-09	Gestión de tareas del proyecto	Como colaborador deseo gestionar las tareas que se me han asignado para organizar mi trabajo.	Colaborador	Media	— Parcialmente terminada	—	Módulo tasks existe como stub. Modelo Tarea definido en schema.
HU-29	Crear y asignar tareas a colaboradores	Como líder deseo crear tareas y asignarlas a los miembros del equipo.	Líder	Media	— No iniciada	—	
HU-30	Visualizar tablero Kanban de tareas	Como colaborador deseo ver las tareas en un tablero Kanban para tener visibilidad del avance.	Colaborador	Media	— No iniciada	—	
HU-31	Comentar en tareas	Como colaborador deseo agregar comentarios en las tareas para comunicar avances.	Colaborador	Baja	— No iniciada	—	
HU-32	Seguimiento de avance general	Como líder deseo visualizar el porcentaje de avance del proyecto según tareas completadas.	Líder	Media	— No iniciada	—	
Evidencias y Validación							
HU-10	Subir evidencias de trabajo	Como colaborador deseo subir evidencias para documentar mi participación en el proyecto.	Colaborador	Baja	— Parcialmente terminada	—	Módulo evidence existe como stub.
HU-11	Validar horas de participación	Como coordinador deseo validar las horas reportadas por los estudiantes para beca y extensión.	Coordinador	Baja	— No iniciada	—	
HU-33	Revisar evidencias subidas	Como líder deseo revisar y aprobar o rechazar las evidencias enviadas por los colaboradores.	Líder	Baja	— No iniciada	—	
HU-34	Generar certificado de participación	Como estudiante deseo generar un certificado digital que avale mi participación en un proyecto.	Estudiante	Baja	— No iniciada	—	
HU-35	Exportar historial de participación	Como estudiante deseo exportar un resumen en PDF de mis proyectos y horas de participación.	Estudiante	Baja	— No iniciada	—	

Organizaciones Estudiantiles							
HU-36	Crear organización estudiantil	Como líder deseo registrar una organización estudiantil en la plataforma.	Líder	Media	— Parcialmente terminada	—	<i>Modelo Organización existe en schema con relaciones definidas.</i>
HU-37	Gestionar miembros de organización	Como líder deseo agregar o remover miembros de la organización.	Líder	Media	— No iniciada	—	
HU-38	Ver perfil público de organización	Como estudiante deseo ver el perfil de una organización con sus proyectos activos.	Estudiante	Media	— No iniciada	—	
Comunicación y Notificaciones							
HU-12	Notificaciones generales del sistema	Como usuario deseo recibir notificaciones sobre eventos relevantes del sistema, como cambios de estado de proyectos y nuevas postulaciones, para mantenerme informado.	Estudiante	Media	✓ Completada	Sprint 3	<i>NotificationsService extendido para más eventos. UI de notificaciones mejorada con tipo, agrupación por fecha, página /dashboard/notificaciones y opción de marcar todas como leídas.</i>
HU-22	Notificación al líder de nueva postulación	Como líder deseo recibir una notificación dentro de la plataforma cuando un estudiante se postule a uno de mis proyectos para revisar su solicitud oportunamente.	Líder	Media	✓ Completada	Sprint 3	<i>Notificación automática al creador del proyecto desde POST /postulaciones, incluyendo nombre del postulante y rol solicitado. En Jira Sprint 3 registrado como IESUC-144 / HU-22.</i>
HU-39	Configurar preferencias de notificación	Como usuario deseo configurar qué tipo de notificaciones deseo recibir.	Estudiante	Baja	— No iniciada	—	
HU-40	Mensajería interna entre usuarios	Como usuario deseo enviar y recibir mensajes privados dentro de la plataforma.	Estudiante	Baja	— No iniciada	—	
HU-41	Anuncios dentro de un proyecto	Como líder deseo publicar anuncios visibles para todos los colaboradores del proyecto.	Líder	Baja	— No iniciada	—	
Panel de Administración							
HU-42	Panel de administración general	Como administrador deseo un panel para gestionar usuarios, proyectos y organizaciones.	Administrador	Media	— No iniciada	—	
HU-43	Gestión de catálogos del sistema	Como administrador deseo gestionar los catálogos de carreras, habilidades e intereses.	Administrador	Media	— Parcialmente terminada	—	<i>Módulo catalogs con endpoints GET carreras, habilidades, intereses ya operativos.</i>
HU-44	Moderación de contenido	Como administrador deseo revisar y moderar proyectos u organizaciones reportados.	Administrador	Baja	— No iniciada	—	
HU-45	Gestión de roles de usuario	Como administrador deseo asignar o revocar roles (estudiante, líder, coordinador, admin).	Administrador	Media	— Parcialmente terminada	—	<i>Enum RolUsuario definido en schema.</i>

Reportes y Analítica							
HU-46	Dashboard de métricas para líderes	Como líder deseo ver métricas de mi proyecto: postulaciones recibidas, roles cubiertos, avance.	Líder	Baja	— No iniciada	—	
HU-47	Reporte de horas por estudiante	Como coordinador deseo generar un reporte de horas beca y extensión por estudiante.	Coordinador	Baja	— No iniciada	—	
HU-48	Estadísticas generales de la plataforma	Como administrador deseo visualizar estadísticas globales de uso de la plataforma.	Administrador	Baja	— No iniciada	—	
HU-49	Exportar reportes a PDF	Como líder o administrador deseo exportar reportes en formato PDF.	Líder	Baja	— No iniciada	—	
Portafolio y Reconocimiento							
HU-50	Portafolio público del estudiante	Como estudiante deseo un portafolio público con mis proyectos completados y certificados.	Estudiante	Baja	— No iniciada	—	
HU-51	Historial de proyectos completados	Como estudiante deseo consultar el historial completo de mis proyectos.	Estudiante	Baja	— No iniciada	—	
HU-52	Valoraciones entre colaboradores	Como colaborador deseo calificar mi experiencia en un proyecto finalizado.	Colaborador	Baja	— No iniciada	—	
HU-60	Feedback de actividades del proyecto ★	Como colaborador deseo dar feedback sobre las actividades del proyecto en curso.	Colaborador	Baja	— No iniciada	—	
Social y Descubrimiento							
HU-58	Seguir a creadores de proyectos ★	Como estudiante deseo seguir a líderes para recibir actualizaciones sobre sus nuevas publicaciones.	Estudiante	Baja	— No iniciada	—	
Calendario y Planificación							
HU-53	Calendario de actividades del proyecto	Como colaborador deseo ver un calendario con los hitos y fechas límite del proyecto.	Colaborador	Baja	— No iniciada	—	
HU-54	Recordatorios de fechas límite	Como colaborador deseo recibir recordatorios automáticos antes del vencimiento de tareas.	Colaborador	Baja	— No iniciada	—	
Configuración y Experiencia de Usuario							
HU-55	Tema visual (modo claro/oscuro)	Como usuario deseo alternar entre modo claro y oscuro según mi preferencia.	Estudiante	Media	— No iniciada	—	
HU-56	Diseño responsivo para dispositivos móviles	Como usuario deseo acceder a la plataforma desde celular o tablet con experiencia optimizada.	Estudiante	Media	— No iniciada	—	

HU-57	Tutorial / guía de uso interactiva	Como usuario nuevo deseo acceder a un tutorial o guía interactiva dentro del sistema.	Estudiante	Media	— No iniciada	—	
--------------	------------------------------------	---	------------	-------	---------------	---	--

Sprint Backlog

1. Pila actual del sprint:

Tipo	ID	Título	SP	Hrs Est.	Responsable	Inicio	Fin	Prioridad	Estado
Historia	HU-62	Hosteo en servidor Azure con deploy automatizado9	5	-	Angel Sanabria	28/04	30/04	Alta	Done
Subtarea	T-50	Configuración de Nginx como reverse proxy	2	4	Angel Sanabria	28/04	29/04	Alta	Done
Subtarea	T-51	Verificación del pipeline CI/CD y Prisma en producción	2	3	Angel Sanabria	29/04	30/04	Alta	Done
Subtarea	T-52	Documentación del entorno de producción	1	2	Angel Sanabria	30/04	30/04	Media	Done
Historia	HU-28	Ver equipo actual del proyecto	3	-	Saul Castillo	28/04	02/05	Alta	Done
Subtarea	T-53	Endpoint GET /proyectos/:id/equipo	1	3	Saul Castillo	28/04	30/04	Alta	Done
Subtarea	T-54	Vista del equipo del proyecto en el dashboard	2	4	Saul Castillo	30/04	02/05	Alta	Done
Historia	HU-26	Editar proyecto existente	3	-	Vernel Hernández	28/04	02/05	Media	Done
Subtarea	T-55	Endpoint PATCH /proyectos/:id para editar información	1	2	Vernel Hernández	28/04	30/04	Media	Done
Subtarea	T-56	Formulario de edición de proyecto con datos precargados	2	1	Vernel Hernández	30/04	02/05	Media	Done
Historia	HU-22	Notificación al líder de nueva postulación	2	-	Derek Coronado	29/04	30/04	Media	Done
Subtarea	T-59	Generación automática de notificación al líder al recibir postulación	2	3	Derek Coronado	29/04	30/04	Media	Done
Historia	HU-12	Notificaciones generales del sistema	3	-	Derek Coronado	28/04	02/05	Media	Done
Subtarea	T-57	Extensión del servicio de notificaciones a más eventos	1	3	Derek Coronado	28/04	01/05	Media	Done
Subtarea	T-58	UI mejorada del panel de notificaciones	2	4	Derek Coronado	01/05	02/05	Media	Done
Historia	HU-20	Cancelar postulación enviada	3	-	Samuel Robledo	01/05	03/05	Media	Done
Subtarea	T-60	Endpoint DELETE /postulaciones/:id	1	2	Samuel Robledo	01/05	02/05	Media	Done
Subtarea	T-61	Botón de cancelar en vista Mis Postulaciones	2	3	Samuel Robledo	02/05	03/05	Media	Done
Historia	HU-61	Autocompletar correo con carné o apellido	2	-	Samuel Robledo	30/04	01/05	Media	Done
Subtarea	T-62	Autocompletado de correo institucional en formulario de registro	2	3	Samuel Robledo	30/04	01/05	Media	Done

Historia	HU-59	Ver proyectos destacados	3	-	Saul Castillo	02/05	07/05	Media	Done
Subtarea	T-63	Endpoint GET /proyectos/destacados	1	3	Saul Castillo	02/05	05/05	Media	Done
Subtarea	T-64	Sección de proyectos destacados en el dashboard	2	4	Saul Castillo	05/05	07/05	Media	Done
Historia	HU-14	Recuperación de contraseña	5	-	Angel Sanabria	03/05	07/05	Media	Done
Subtarea	T-65	Endpoints POST /auth/forgot-password y reset-password	3	5	Angel Sanabria	03/05	06/05	Media	Done
Subtarea	T-66	UI de recuperación de contraseña	2	3	Angel Sanabria	06/05	07/05	Media	Done
Historia	HT-03	Tareas transversales Sprint 3	5	-	Equipo	07/05	10/05	Alta	Done
Subtarea	T-67	Pruebas de integración del flujo completo Sprint 3	1	4	Vernel Hernández	07/05	08/05	Alta	Done
Subtarea	T-68	Corrección de errores y ajustes visuales	1	3	Derek Coronado	08/05	09/05	Media	Done
Subtarea	T-69	Revisión de seguridad y validación en producción	1	2	Angel Sanabria	08/05	09/05	Alta	Done
Subtarea	T-70	Preparación de presentación y guion Sprint 3	1	3	Samuel Robledo	09/05	10/05	Media	Done
Subtarea	T-71	Validación del deploy y accesibilidad pública	1	2	Saul Castillo	10/05	10/05	Alta	Done

2. Calendario de planificación del sprint:

Fecha	Angel	Saul	Vernel	Derek	Samuel
23/04 Jue	-	-	-	-	-
24/04 Vie	-	-	-	-	-
25/04 Sáb	-	-	-	-	-
26/04 Dom	-	-	-	-	-
27/04 Lun	Planificación	Planificación	Planificación	Planificación	Planificación
28/04 Mar	T-50	T-53	T-55	T-57	-
29/04 Mié	T-50, T-51	T-53	T-55	T-57, T-59	-
30/04 Jue	T-51, T-52	T-53, T-54	T-55, T-56	T-57, T-59	T-62
01/05 Vie	-	T-54	T-56	T-57, T-58	T-60, T-62
02/05 Sáb	-	T-54, T-63	T-56	T-58	T-60, T-61
03/05 Dom	T-65	T-63	-	-	T-61
04/05 Lun	T-65	T-63	-	-	-
05/05 Mar	T-65	T-63, T-64	-	-	-
06/05 Mié	T-65, T-66	T-64	-	-	-
07/05 Jue	T-66	T-64	T-67	-	-
08/05 Vie	T-69	-	T-67	T-68	-
09/05 Sáb	T-69	-	-	T-68	T-70
10/05 Dom	Revisión	T-71, Revisión	Revisión	Revisión	T-70, Revisión
11/05 Lun	Revisión final	Revisión final	Revisión final	Revisión final	Revisión final

Incremento

Vínculo al repositorio

Repositorio: <https://github.com/asanabria-2021067/proyecto-ingenieria-software>

El código desarrollado y funcional del Sprint 3 se encuentra integrado en la rama principal (origin/main) del repositorio indicado. Este incremento reúne funcionalidades de infraestructura y despliegue en Azure, gestión de proyectos, postulaciones, notificaciones extendidas, recuperación de contraseña, autocompletado de correo institucional, proyectos destacados y mejoras transversales de validación y seguridad.

Nota sobre la revisión del historial de commits: Se recomienda revisar tanto la rama main como la rama develop del repositorio. Al realizarse el merge hacia main, los cambios se consolidaron como un único commit de merge, por lo que el historial visible en la interfaz web de GitHub no refleja de forma individual los commits realizados por cada integrante del equipo durante el sprint. Sin embargo, al inspeccionar el repositorio mediante comandos de Git en terminal (por ejemplo, `git log --all --oneline --graph` o `git log develop`), sí es posible visualizar el historial completo de commits con sus respectivos autores, fechas y mensajes. Se solicita considerar esta particularidad al momento de evaluar la contribución individual de cada miembro del equipo.

Aplicación hosteada

La aplicación se encuentra desplegada en un servidor Azure con IP pública **158.23.57.118**. El acceso se realiza mediante <http://158.23.57.118/>, donde Nginx actúa como reverse proxy enrutando las peticiones: la ruta raíz (/) sirve el frontend construido con Next.js, y las peticiones a /api/* son redirigidas al backend NestJS. La configuración exacta de Nginx no se encuentra versionada dentro del repositorio; sin embargo, el README documenta el flujo esperado del reverse proxy y los puertos involucrados.

Stack Tecnológico

- ☐ NestJS 11
- ☐ Prisma 6.19.2
- ☐ PostgreSQL 17 (Alpine)
- ☐ Next.js 15.1.0
- ☐ React 19.0.0
- ☐ TypeScript 5.7.0
- ☐ JWT + bcrypt
- ☐ Tailwind CSS
- ☐ Radix UI (Material Design 3)
- ☐ Node.js 22
- ☐ Docker / Docker Compose
- ☐ GitHub Actions (CI/CD)
- ☐ Azure VM (producción)

Estructura del proyecto

```
proyecto-ingenieria-software/  
  apps/backend/           → API NestJS + Prisma + PostgreSQL  
  apps/frontend/         → Next.js 15 + React 19 + TypeScript  
  docker-compose.yml      → Orquestación de servicios  
  docker-compose.dev.yml  → Configuración de desarrollo con hot-reload  
  .github/workflows/      → Pipeline de despliegue automatizado (deploy.yml)  
  .env.example            → Variables de entorno de referencia  
  Corte 3/                → Documentación académica del sprint
```

Estado del código en la rama principal

El repositorio está organizado como un monorepo que separa de forma clara las aplicaciones de backend y frontend, junto con las carpetas de entregables académicos correspondientes a cada corte del curso. La estructura general comprende los directorios apps/backend (API construida en NestJS con Prisma y PostgreSQL), apps/frontend (aplicación construida en Next.js 15 con React 19 y TypeScript), las carpetas Corte 1, Corte 2, Corte 3, Avances 1, Avances 2 y Scrum para la documentación, así como los archivos de orquestación de Docker en la raíz del proyecto.

El backend se encuentra organizado por módulos: auth, projects, applications (postulaciones), notifications, users, catalogs, prisma, evidence, tasks y validation. El frontend incluye rutas del dashboard, edición de proyecto, vista de equipo, notificaciones completas, registro con autocompletado, recuperación de contraseña, proyectos destacados y las vistas previamente construidas en Sprint 1 y Sprint 2. El Sprint 3 amplió funcionalidades ya construidas en iteraciones anteriores y agregó la capa de infraestructura necesaria para el despliegue en producción.

Documentación de los archivos de Docker e infraestructura

La rama principal incluye los archivos necesarios para levantar el sistema mediante Docker y desplegar la aplicación en producción. A continuación se describen los archivos presentes y su propósito.

1. docker-compose.yml (raíz del proyecto)

Define la orquestación completa del sistema. Incluye los servicios postgres (importado desde apps/backend/docker-compose.yml), backend y frontend, asociados al perfil app y conectados mediante la red uvg-network. El backend depende de PostgreSQL mediante healthcheck y el frontend depende del backend, garantizando un orden de arranque correcto.

```
// ruta: docker-compose.yml  
services:  
  backend:  
    build:  
      context: ./apps/backend  
      dockerfile: Dockerfile  
      container_name: uvg-collab-backend  
      profiles: [app]
```

```

ports: ["3001:3001"]
environment:
  - DATABASE_URL=postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/${DB_NAME}
  - JWT_SECRET=${JWT_SECRET:-dev-secret-change-me}
depends_on:
  postgres:
    condition: service_healthy
networks: [uvg-network]

frontend:
  build:
    context: ./apps/frontend
  args:
    - NEXT_PUBLIC_API_URL=${NEXT_PUBLIC_API_URL:-http://localhost:3001}
  ports: ["3000:3000"]
  depends_on: [backend]
  networks: [uvg-network]

```

2. apps/backend/src/main.ts

El punto de entrada del backend configura el prefijo global /api para todas las rutas, habilita la validación global de DTOs mediante ValidationPipe, y activa CORS. Este prefijo es esencial para el correcto funcionamiento del reverse proxy en producción.

```

// ruta: apps/backend/src/main.ts - líneas 5-18
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.setGlobalPrefix('api');
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    })
  );
  app.enableCors({ origin: true, credentials: true });
  const port = process.env.PORT ?? 3001;
  await app.listen(port);
  console.log(`Backend running on port ${port}`);
}

```

3. .github/workflows/deploy.yml

El pipeline de despliegue automatizado se activa en cada push a la rama main o manualmente mediante workflow_dispatch. Utiliza appleboy/ssh-action para conectarse al servidor Azure vía SSH, actualizar el código, reconstruir los contenedores Docker y ejecutar un health check que verifica la disponibilidad del backend en el puerto 3001. Finalmente ejecuta el seed de Prisma para poblar datos iniciales.

```

// ruta: .github/workflows/deploy.yml - extracto del health check
for i in $(seq 1 60); do
  if curl -fsS http://127.0.0.1:3001/api >/dev/null; then
    echo "Backend is healthy."
  fi
done

```



```
    break
  fi
  if [ "$i" -eq 60 ]; then
    echo "Backend did not become healthy in time"
    exit 1
  fi
  sleep 5
done

docker compose exec -T backend npx prisma db seed
docker compose --profile app ps
docker image prune -f
```

4. .env.example

Lista todas las variables de entorno requeridas para la ejecución del sistema, sin incluir valores sensibles. Sirve como referencia para configurar el archivo .env tanto en desarrollo como en producción.

```
// ruta: .env.example
NODE_ENV=development
DB_USER=postgres
DB_PASSWORD=postgres
DB_NAME=uvg_collab
DB_PORT=5433
JWT_SECRET=super-secret-key-change-in-production
DATABASE_URL=postgresql://postgres:postgres@localhost:5433/uvg_collab?schema=public
NEXT_PUBLIC_API_URL=http://localhost:3001
NEXT_PUBLIC_API_PREFIX=/api
```

5. README.md — Flujo de producción

El README documenta la URL pública del sistema y el diagrama de enrutamiento del reverse proxy, proporcionando una referencia rápida sobre cómo se accede a la aplicación en producción.

```
// ruta: README.md
### Flujo en produccion (reverse-proxy)
- / -> Frontend (Next.js)
- /api/* -> Backend (NestJS)
IP publica: 158.23.57.118

### URL publica (hosteado)
- Aplicacion: http://158.23.57.118/
```

Funcionalidades planificadas que se completaron

A continuación se documenta cada una de las diez historias de usuario y técnicas completadas durante el Sprint 3, describiendo la implementación realizada, los archivos principales involucrados y los fragmentos de código relevantes extraídos del repositorio.

1. HU-62: Hosteo en servidor Azure con deploy automatizado

Esta historia cubrió el despliegue completo de la aplicación en un servidor Azure con pipeline de CI/CD automatizado. El trabajo se distribuyó en tres subtarefas: configuración de Nginx como reverse proxy (T-50), verificación del pipeline CI/CD y Prisma en producción (T-51) y documentación del entorno de producción (T-52).

En el backend se agregó `app.setGlobalPrefix('api')` en `main.ts` para que todas las rutas respondan bajo el prefijo `/api`, permitiendo que Nginx distinga entre peticiones al frontend y al backend. El archivo `docker-compose.yml` expone el backend en el puerto 3001 y el frontend en el 3000, mientras que Nginx (configurado directamente en la VM de Azure) escucha en el puerto 80 y enruta las peticiones según la ruta solicitada.

El pipeline de GitHub Actions (`deploy.yml`) ejecuta el deploy completo: actualiza el repositorio en el servidor, reconstruye los contenedores Docker, verifica la salud del backend con hasta 60 reintentos y ejecuta el seed de Prisma. Los secrets del pipeline incluyen `SERVER_IP`, `SERVER_USER`, `SSH_PRIVATE_KEY`, `DB_USER`, `DB_PASSWORD`, `JWT_SECRET`, `CLOUDINARY_CLOUD_NAME` y `CLOUDINARY_UPLOAD_PRESET`.

Limitaciones detectadas: la configuración de Nginx no está versionada en el repositorio, lo que impide auditar los bloques `location` directamente desde el código fuente. Adicionalmente, el pipeline no ejecuta `prisma migrate deploy`, por lo que las migraciones de base de datos deben aplicarse manualmente en el servidor.

2. HT-03: Tareas transversales Sprint 3

Esta historia técnica agrupó cinco subtarefas de carácter transversal: pruebas de integración (T-67), corrección de errores y ajustes visuales (T-68), revisión de seguridad (T-69), preparación de la presentación (T-70) y validación del deploy (T-71).

Pruebas de integración (T-67): Se implementaron tests unitarios con Vitest para los servicios principales del backend (`AuthService`, `ProjectsService`, `ApplicationsService`, `NotificationsService`, `JwtStrategy`) y tests de flujos simulados en el frontend con `apiFetch` mockeado. No se implementó E2E automatizado con herramientas como Cypress o Playwright.

```
// ruta: apps/frontend/test/user-flows.spec.ts - líneas 10-46
describe('simulación de procesos de usuario', () => {
  it('flujo líder: login -> crear borrador -> enviar revisión -> cierre', async () => {
    (apiFetch as any)
      .mockResolvedValueOnce({ accessToken: 'jwt-1' })
      .mockResolvedValueOnce({ idProyecto: 10, estadoProyecto: 'BORRADOR' })
      .mockResolvedValueOnce({ idProyecto: 10, estadoProyecto: 'EN_REVISION' })
      .mockResolvedValueOnce({ idProyecto: 10, estadoProyecto: 'EN_SOLICITUD_CIERRE' });

    const auth = await login({ correo: 'lider@uvg.edu', contrasena: '123456' });
    const created = await createProject({ tituloProyecto: 'X' } as any);
    const submitted = await submitProjectForReview(created.idProyecto);
    const closeReq = await requestProjectClosure(submitted.idProyecto);

    expect(auth.accessToken).toBe('jwt-1');
    expect(closeReq.estadoProyecto).toBe('EN_SOLICITUD_CIERRE');
  });
});
```

Revisión de seguridad (T-69): Se verificó que todos los endpoints nuevos del Sprint 3 están protegidos con JwtAuthGuard. La estrategia JWT extrae el Bearer token del header Authorization con ignoreExpiration: false. Los únicos endpoints sin guard son POST /auth/forgot-password y POST /auth/reset-password, lo cual es correcto por diseño al ser endpoints públicos.

```
// ruta: apps/backend/src/auth/jwt.strategy.ts - Líneas 6-18
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: process.env.JWT_SECRET || 'dev-secret-change-me',
    });
  }

  validate(payload: { sub: number; correo: string }) {
    return { userId: payload.sub, correo: payload.correo };
  }
}
```

Corrección de errores (T-68): Todas las páginas nuevas del sprint implementan estados de carga (isLoading), error (isError) y vacío. Se agregó modal de confirmación vía SweetAlert2 para la cancelación de postulaciones.

Preparación de presentación (T-70): Las diapositivas y el guion fueron elaborados en Canva. Esta evidencia no se encuentra en el repositorio y requiere verificación externa.

Validación del deploy (T-71): Se confirmó la accesibilidad pública del sistema en la IP 158.23.57.118. El pipeline ejecuta un health check automático contra el endpoint /api del backend.

3. HU-26: Editar proyecto existente

Se implementó la funcionalidad que permite al líder creador de un proyecto editar su información, distribuida en dos subtarefas: endpoint PATCH /proyectos/:id (T-55) y formulario de edición con datos precargados (T-56).

En el backend se registraron tanto @Put('/:id') como @Patch('/:id'), ambos protegidos con JwtAuthGuard y ambos delegando al mismo método update() del servicio. Este método invoca _requireOwner() que lanza ForbiddenException si el usuario autenticado no es el creador del proyecto. La edición solo está habilitada para proyectos en estado BORRADOR u OBSERVADO.

```
// ruta: apps/backend/src/projects/projects.service.ts - Líneas 23-26, 501-507
const ESTADOS_EDITABLES: EstadoProyecto[] = [
  EstadoProyecto.BORRADOR,
  EstadoProyecto.OBSERVADO,
];

async update(id: number, data: UpdateProjectDto, userId: number) {
  const proyecto = await this._requireOwner(id, userId);
  if (!ESTADOS_EDITABLES.includes(proyecto.estadoProyecto)) {
    throw new BadRequestException(
      `Solo se puede editar un proyecto en estado ${ESTADOS_EDITABLES.join(' o ')}`,
    );
  }
}
```

```
// ruta: apps/backend/src/projects/projects.service.ts – líneas 809-820
private async _requireOwner(idProyecto: number, userId: number) {
  const proyecto = await this.prisma.proyecto.findFirst({
    where: { idProyecto, eliminadoEn: null },
    select: { idProyecto: true, estadoProyecto: true, creadoPor: true },
  });
  if (!proyecto) {
    throw new NotFoundException(`Proyecto con id ${idProyecto} no encontrado`);
  }
  if (proyecto.creadoPor !== userId) {
    throw new ForbiddenException('No eres el líder de este proyecto');
  }
  return proyecto;
}
```

En el frontend, la ruta `/dashboard/proyectos/[id]/editar` redirige al formulario compartido de creación en modo edición, detectando el parámetro `editId` desde los query params. Los datos del proyecto se precargan mediante `useQuery` y `useEffect`, incluyendo roles y requisitos. Al guardar, el servicio invoca `updateProject()` con método PUT.

```
// ruta: apps/frontend/app/dashboard/projects/mine/form/page.tsx – líneas 42-77 (extracto)
const { data: proyectoExistente, isLoading: loadingExistente } = useQuery({
  queryKey: ['proyecto-owner', editId],
  queryFn: () => getMyProjectById(editId!),
  enabled: editId !== null,
});

useEffect(() => {
  if (!proyectoExistente) return;
  const p = proyectoExistente;
  setForm({
    tituloProyecto: p.tituloProyecto,
    descripcionProyecto: p.descripcionProyecto ?? '',
    tipoProyecto: p.tipoProyecto as TipoProyecto,
    modalidadProyecto: p.modalidadProyecto as ModalidadProyecto,
    fechaInicio: p.fechaInicio ? p.fechaInicio.split('T')[0] : '',
    fechaFinEstimada: p.fechaFinEstimada ? p.fechaFinEstimada.split('T')[0] : '',
    roles: p.roles.map((r) => ({ /* ... mapeo de roles y requisitos */ })),
  });
}, [proyectoExistente]);
```

Observación: el servicio frontend utiliza PUT en lugar de PATCH. Esto es funcionalmente equivalente porque el backend registra ambos métodos apuntando al mismo servicio `update()`.

4. HU-22: Notificación al líder de nueva postulación

Se implementó la generación automática de notificaciones al líder del proyecto cuando un estudiante se postula a un rol (T-59). El `NotificationsService` fue inyectado en el constructor de `ApplicationsService`, y al crear una postulación exitosa se invoca `notifyUsers()` con el ID del creador del proyecto como destinatario.

```
// ruta: apps/backend/src/applications/applications.service.ts – líneas 100-113
// Notificar al líder del proyecto
await this.notificationsService.notifyUsers(
  [rol.proyecto.creadoPor],
```

```

{
  tipoNotificacion: 'NUEVA_POSTULACION',
  tituloNotificacion: 'Nueva postulación recibida',
  mensajeNotificacion: `${postulacion.postulante.nombre}
${postulacion.postulante.apellido} se postuló para el rol "${rol.nombreRol}" en tu proyecto
"${rol.proyecto.tituloProyecto}".`,
  datosJson: {
    idPostulacion: postulacion.idPostulacion,
    idProyecto: rol.proyecto.idProyecto,
    idRolProyecto: dto.idRolProyecto,
  },
},
);

```

La notificación incluye el nombre completo del postulante, el nombre del rol solicitado y el título del proyecto, proporcionando al líder toda la información necesaria para revisar la solicitud oportunamente.

5. HU-12: Notificaciones generales del sistema

Esta historia extendió el módulo de notificaciones construido en el Sprint 2 para cubrir más eventos del sistema y mejorar la interfaz de usuario. Se distribuyó en dos subtarear: extensión del servicio a más eventos (T-57) y UI mejorada del panel de notificaciones (T-58).

En el backend, el método `changeEstado()` del servicio de proyectos ahora invoca `notifyProjectActiveParticipants()` para los estados `PUBLICADO`, `EN_PROGRESO` y `CERRADO`, generando notificaciones de tipo `CAMBIO_ESTADO_PROYECTO` para todos los participantes activos del proyecto excepto el autor de la acción.

```

// ruta: apps/backend/src/projects/projects.service.ts - Líneas 787-807
if (
  nuevoEstado === EstadoProyectoCreador.PUBLICADO ||
  nuevoEstado === EstadoProyectoCreador.EN_PROGRESO ||
  nuevoEstado === EstadoProyectoCreador.CERRADO
) {
  await this.notifications.notifyProjectActiveParticipants(
    id,
    userId,
    {
      tipoNotificacion: 'CAMBIO_ESTADO_PROYECTO',
      tituloNotificacion: `Proyecto cambió a ${nuevoEstado}`,
      mensajeNotificacion: `El proyecto "${actualizado.tituloProyecto}" cambió su estado a
${nuevoEstado}.`,
      datosJson: { idProyecto: id, nuevoEstado },
    },
  );
}

```

En el frontend se implementó la página completa `/dashboard/notificaciones` con agrupación por fecha, estados de carga y vacío, y botón de “Marcar todas como leídas”. El componente de campana (`notifications-bell.tsx`) muestra el conteo de notificaciones no leídas y permite marcar todas como leídas mediante `PATCH /notificaciones/leer-todas`.

```

// ruta: apps/frontend/app/dashboard/notificaciones/page.tsx - Líneas 57-69

```

```
const groupByDate = (notifs: Notificacion[]) => {
  const groups: Record<string, Notificacion[]> = {};
  notifs.forEach((n) => {
    const fecha = new Date(n.creadaEn).toLocaleDateString('es-GT', {
      year: 'numeric',
      month: 'long',
      day: 'numeric',
    });
    if (!groups[fecha]) groups[fecha] = [];
    groups[fecha].push(n);
  });
  return groups;
};
```

Limitación detectada: la página /dashboard/notificaciones utiliza POST /notificaciones/marcar-todas en su mutationFn, mientras que el backend expone PATCH /notificaciones/leer-todas. El hook del componente de campana sí utiliza el endpoint correcto.

6. HU-28: Ver equipo actual del proyecto

Se implementó la funcionalidad para que el líder pueda visualizar la lista de colaboradores aceptados en su proyecto, distribuida en dos subtarear: endpoint GET /proyectos/:id/equipo (T-53) y vista del equipo en el dashboard (T-54).

El endpoint consulta la tabla participacionProyecto filtrando por estadoParticipacion: 'ACTIVO' y devuelve nombre, apellido, correo, carrera, habilidades, rol asignado y fecha de inicio para cada colaborador. El endpoint está protegido con JwtAuthGuard.

```
// ruta: apps/backend/src/projects/projects.service.ts – líneas 298-353 (extracto)
async findTeam(id: number) {
  const proyecto = await this.prisma.proyecto.findFirst({
    where: { idProyecto: id, eliminadoEn: null },
    select: { idProyecto: true },
  });
  if (!proyecto) {
    throw new NotFoundException(`Proyecto con id ${id} no encontrado`);
  }

  const participaciones = await this.prisma.participacionProyecto.findMany({
    where: {
      rolProyecto: { idProyecto: id },
      estadoParticipacion: 'ACTIVO',
    },
    select: {
      idParticipacionProyecto: true,
      fechaInicio: true,
      rolProyecto: { select: { idRolProyecto: true, nombreRol: true } },
      usuario: {
        select: {
          idUsuario: true, nombre: true, apellido: true, correo: true,
          perfil: { select: { carrera: { select: { nombreCarrera: true } } } },
          habilidades: {
            select: { habilidad: { select: { idHabilidad: true, nombreHabilidad: true } } },
          },
        },
      },
    },
  });
}
```

```

    },
    orderBy: { fechaInicio: 'asc' },
  });
  return participaciones;
}

```

En el frontend, la página `/dashboard/proyectos/[id]/equipo` muestra tarjetas con la información de cada colaborador, incluyendo el rol como chip destacado y las habilidades como badges.

Limitación detectada: el endpoint no valida que el usuario autenticado sea el líder del proyecto; cualquier usuario autenticado puede consultar el equipo de cualquier proyecto.

7. HU-20: Cancelar postulación enviada

Se implementó la funcionalidad para que un estudiante pueda cancelar una postulación en estado PENDIENTE, distribuida en dos subtarefas: endpoint DELETE `/postulaciones/:id` (T-60) y botón de cancelar en la vista Mis Postulaciones (T-61).

El servicio `delete()` valida la existencia de la postulación, verifica que el usuario autenticado sea el dueño (`idUsuarioPostulante === userId`) y confirma que el estado sea PENDIENTE antes de ejecutar un `hard delete`.

```

// ruta: apps/backend/src/applications/applications.service.ts - líneas 257-288
async delete(id: number, userId: number) {
  const postulacion = await this.prisma.postulacion.findUnique({
    where: { idPostulacion: id },
    select: {
      idPostulacion: true,
      idUsuarioPostulante: true,
      estadoPostulacion: true,
    },
  });

  if (!postulacion) {
    throw new NotFoundException(`Postulación con id ${id} no encontrada`);
  }
  if (postulacion.idUsuarioPostulante !== userId) {
    throw new ForbiddenException('No tienes permiso para cancelar esta postulación');
  }
  if (postulacion.estadoPostulacion !== 'PENDIENTE') {
    throw new BadRequestException('Solo puedes cancelar postulaciones en estado PENDIENTE');
  }

  await this.prisma.postulacion.delete({ where: { idPostulacion: id } });
  return { mensaje: 'Postulación cancelada exitosamente' };
}

```

En el frontend, el botón “Cancelar” se muestra únicamente cuando la postulación está en estado PENDIENTE. Al hacer clic, se presenta un modal de confirmación vía `SweetAlert2` antes de ejecutar la eliminación. Tras la operación exitosa, la lista se refresca automáticamente mediante invalidación de queries.

```

// ruta: apps/frontend/app/dashboard/mis-postulaciones/page.tsx - líneas 167-175
{p.estadoPostulacion === 'PENDIENTE' && (

```

```

<button
  onClick={() => handleCancelar(p.idPostulacion)}
  disabled={deleteMutation.isPending}
  className="inline-flex items-center gap-1 px-3 py-1.5 rounded-lg
    bg-error/10 text-error text-xs font-semibold
    hover:bg-error/20 transition-colors disabled:opacity-50"
  >
  <Trash2 className="w-3.5 h-3.5" />
  Cancelar
</button>
)}

```

8. HU-61: Autocompletar correo con carné o apellido

Se implementó la lógica de autocompletado del correo institucional en el formulario de registro (T-62). Mediante un `useEffect` que observa el campo de carné, el sistema genera automáticamente el correo `@uvg.edu.gt` cuando el usuario ingresa su número de carné, siempre que el campo de correo no contenga ya un carácter `@`.

```

// ruta: apps/frontend/app/registro/page.tsx - Líneas 33-36
useEffect(() => {
  if (carne && carne.trim() !== '' && !correo.includes('@')) {
    setCorreo(`${carne.trim()}@uvg.edu.gt`);
  }
}, [carne]);

```

Adicionalmente, la validación en el submit rechaza correos que no terminen en `@uvg.edu.gt`, asegurando que solo se registren correos institucionales válidos.

```

// ruta: apps/frontend/app/registro/page.tsx - Líneas 44-46
if (!correo.endsWith('@uvg.edu.gt')) {
  uvgSwal.fire({ icon: 'warning', title: 'Error', text: 'El correo debe ser @uvg.edu.gt' });
  return;
}

```

Observación: la implementación actual autocompleta únicamente a partir del carné, no del apellido. El título de la historia menciona ambas opciones, pero la lógica implementada se limita al carné.

9. HU-59: Ver proyectos destacados

Se implementó la funcionalidad de proyectos destacados, distribuida en dos subtarear: endpoint `GET /proyectos/destacados` (T-63) y sección visual en el dashboard (T-64).

El endpoint es público (sin guard), filtra proyectos en estado `PUBLICADO` con `eliminadoEn: null`, los ordena por cantidad de postulaciones de forma descendente y devuelve un máximo de 6 resultados.

```

// ruta: apps/backend/src/projects/projects.service.ts - Líneas 355-376
async findFeatured() {
  const proyectos = await this.prisma.proyecto.findMany({
    where: {
      estadoProyecto: EstadoProyecto.PUBLICADO,
      eliminadoEn: null,
    },
  },

```



```

    select: {
      ...proyectoListSelect,
      _count: {
        select: { roles: { where: { postulaciones: { some: {} } } } },
      },
    },
    orderBy: {
      roles: { _count: 'desc' },
    },
    take: 6,
  });
  return proyectos;
}

```

En el frontend, la sección “Proyectos Destacados” se agregó en la página principal del dashboard con un grid de tarjetas y un enlace “Ver todos” que redirige a </dashboard/proyectos>.

Limitación detectada: `apiFetch` no está importado en `dashboard/page.tsx`. El archivo importa `searchProjects` desde el servicio de proyectos pero no importa `apiFetch` desde `@/lib/api/client`, lo que genera un `ReferenceError` en runtime al intentar cargar los proyectos destacados.

10. HU-14: Recuperación de contraseña

Se implementó el flujo completo de recuperación de contraseña, distribuido en dos subtarear: endpoints `POST /auth/forgot-password` y `POST /auth/reset-password` (T-65), y UI de recuperación de contraseña (T-66).

El endpoint `forgot-password` busca al usuario por correo, genera un token JWT con `expirationIn: '1h'` y `payload { tipo: 'reset' }`, y devuelve un mensaje genérico independientemente de si el correo existe (para no revelar información). El endpoint `reset-password` valida el token con `jwtService.verify()`, confirma que el tipo sea `'reset'`, y actualiza la contraseña con `bcrypt.hash(nuevaContraseña, 10)`.

```

// ruta: apps/backend/src/auth/auth.service.ts – líneas 80-104
async forgotPassword(correo: string) {
  const usuario = await this.prisma.usuario.findUnique({
    where: { correo },
    select: { idUsuario: true, correo: true },
  });

  if (!usuario) {
    return {
      mensaje: 'Si el correo existe, recibirás un enlace de recuperación',
    };
  }

  const resetToken = this.jwtService.sign(
    { sub: usuario.idUsuario, correo: usuario.correo, tipo: 'reset' },
    { expiresIn: '1h' },
  );

  // TODO: En producción, enviar email con el token
  return {
    mensaje: 'Si el correo existe, recibirás un enlace de recuperación',
    resetToken, // Solo para desarrollo
  };
}

```

```
// ruta: apps/backend/src/auth/auth.service.ts - Líneas 106-134
async resetPassword(token: string, nuevaContraseña: string) {
  let payload: any;
  try {
    payload = this.jwtService.verify(token);
  } catch (error) {
    throw new BadRequestException('Token inválido o expirado');
  }

  if (payload.tipo !== 'reset') {
    throw new BadRequestException('Token no válido para esta operación');
  }

  const usuario = await this.prisma.usuario.findUnique({
    where: { idUsuario: payload.sub },
  });
  if (!usuario) {
    throw new NotFoundException('Usuario no encontrado');
  }

  const contraseñaHash = await bcrypt.hash(nuevaContraseña, 10);
  await this.prisma.usuario.update({
    where: { idUsuario: usuario.idUsuario },
    data: { contraseña: contraseñaHash },
  });

  return { mensaje: 'Contraseña actualizada exitosamente' };
}
```

En el frontend, la ruta /recuperar-contraseña presenta un formulario de correo que envía POST a /auth/forget-password. En modo desarrollo, muestra el token generado en un SweetAlert con enlace directo a la página de restablecimiento. La ruta /reset-password lee el token desde los query params, valida que las contraseñas coincidan y tengan al menos 8 caracteres, y envía POST a /auth/reset-password. Tras el éxito, redirige al login después de 3 segundos.

Limitación detectada: el servicio de correo no envía emails reales. Existe un comentario TODO en el código indicando que en producción se debería integrar un servicio de email. Actualmente el token se devuelve directamente en la respuesta JSON para propósitos de desarrollo.

Observaciones y limitaciones del incremento

Durante el desarrollo del Sprint 3 se identificaron los siguientes puntos pendientes de fortalecimiento, detectados mediante la auditoría técnica del código fuente:

- ☐ La configuración de Nginx no está versionada en el repositorio; existe únicamente en la VM de Azure.
- ☐ El pipeline CI/CD no ejecuta prisma migrate deploy, lo que requiere aplicación manual de migraciones en producción.
- ☐ El servicio de recuperación de contraseña no envía correos reales; el token se devuelve en la respuesta JSON (modo desarrollo).
- ☐ La sección de proyectos destacados en el dashboard puede presentar error en runtime por falta del import de apiFetch.

- ❑ La página /dashboard/notificaciones utiliza un endpoint incorrecto (POST /notificaciones/marcar-todas) para marcar todas como leídas.
- ❑ El endpoint GET /proyectos/:id/equipo no valida que el solicitante sea el líder del proyecto.
- ❑ No se implementaron pruebas E2E automatizadas; las pruebas existentes son unitarias con mocks.
- ❑ CORS está configurado con origen: true, aceptando peticiones desde cualquier origen.
- ❑ La preparación de la presentación (Canva) no se encuentra evidenciada en el repositorio.

Cierre del incremento

El Sprint 3 representó un avance significativo tanto en funcionalidades de producto como en preparación de infraestructura para producción. Se logró desplegar la aplicación en un servidor Azure con pipeline de CI/CD automatizado, se extendió el sistema de notificaciones para cubrir más eventos del ciclo de vida de los proyectos, y se agregaron flujos importantes para líderes y estudiantes como la edición de proyectos, la vista del equipo, la cancelación de postulaciones, los proyectos destacados y la recuperación de contraseña.

La arquitectura modular del proyecto se mantuvo consistente con las iteraciones anteriores, separando claramente las responsabilidades entre backend y frontend, y aprovechando los patrones establecidos en Sprint 1 y Sprint 2. Aunque existen detalles pendientes de fortalecimiento — particularmente en la integración del servicio de correo y la versionación de la configuración de infraestructura —, el incremento del Sprint 3 representa una mejora clara respecto a las entregas anteriores y acerca la plataforma UVG Collab.

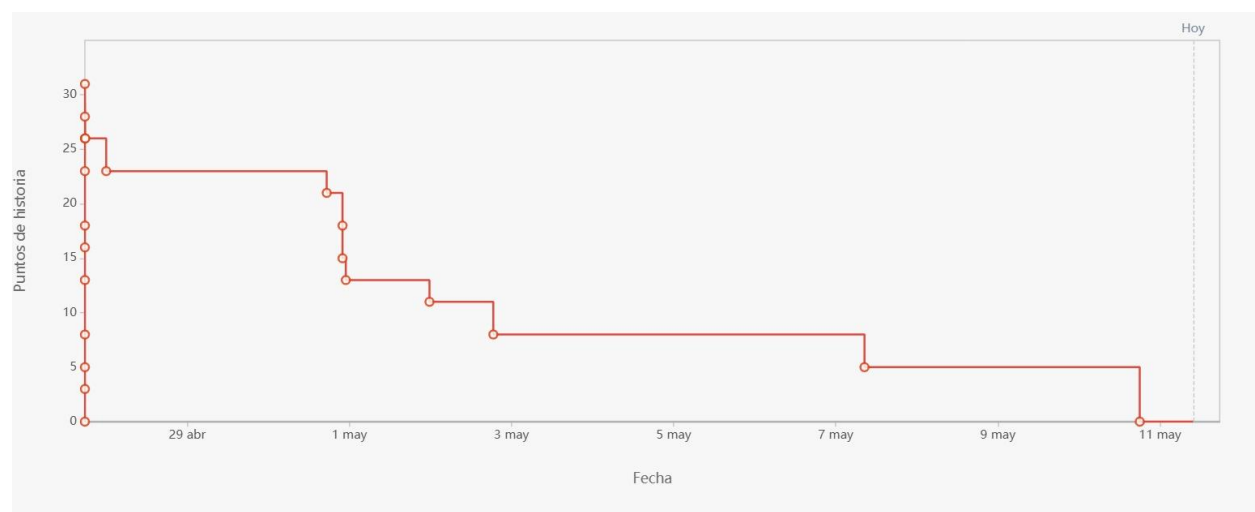
Resultados del Sprint

ID	Tarea	Responsable	Estado
HU-62	Hosteo en servidor Azure con deploy automatizado	Angel Sanabria	Completada
T-50	Configuración de Nginx como reverse proxy	Angel Sanabria	Completada
T-51	Verificación del pipeline CI/CD y Prisma en producción	Angel Sanabria	Completada
T-52	Documentación del entorno de producción	Angel Sanabria	Completada
HT-03	Tareas transversales Sprint 3	Equipo	Completada
T-67	Pruebas de integración del flujo completo Sprint 3	Vernel Hernández	Completada
T-68	Corrección de errores y ajustes visuales	Derek Coronado	Completada
T-69	Revisión de seguridad y validación en producción	Angel Sanabria	Completada
T-70	Preparación de presentación y guion Sprint 3	Samuel Robledo	Completada
T-71	Validación del deploy y accesibilidad pública	Saul Castillo	Completada
HU-26	Editar proyecto existente	Vernel Hernández	Completada
T-55	Endpoint PATCH /proyectos/:id para editar información	Vernel Hernández	Completada
T-56	Formulario de edición de proyecto con datos precargados	Vernel Hernández	Completada
HU-22	Notificación al líder de nueva postulación	Derek Coronado	Completada

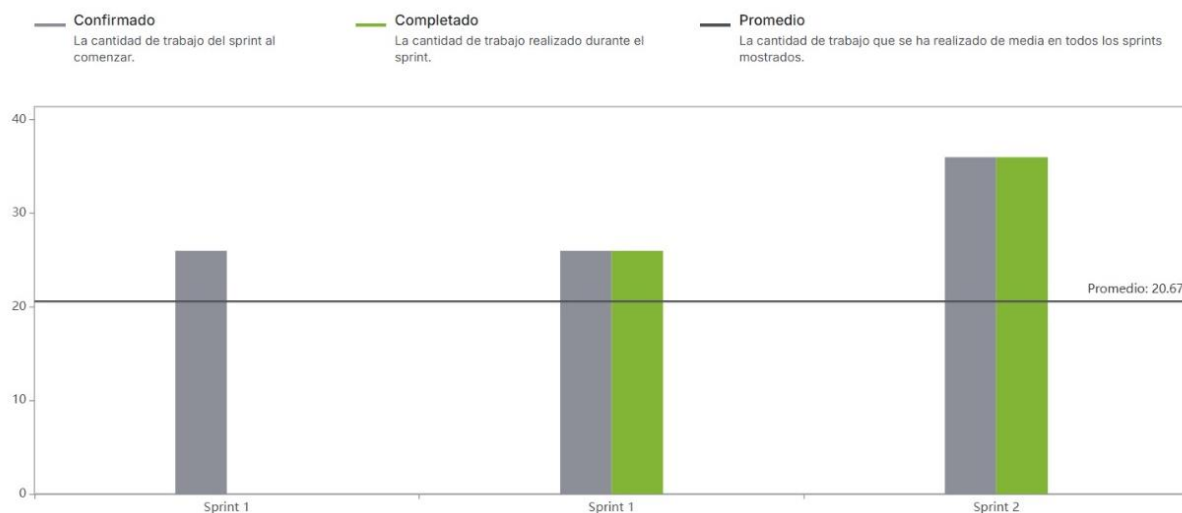
T-59	Generación automática de notificación al líder al recibir postulación	Derek Coronado	Completada
HU-12	Notificaciones generales del sistema	Derek Coronado	Completada
T-57	Extensión del servicio de notificaciones a más eventos	Derek Coronado	Completada
T-58	UI mejorada del panel de notificaciones	Derek Coronado	Completada
HU-28	Ver equipo actual del proyecto	Saul Castillo	Completada
T-53	Endpoint GET /proyectos/:id/equipo	Saul Castillo	Completada
T-54	Vista del equipo del proyecto en el dashboard	Saul Castillo	Completada
HU-20	Cancelar postulación enviada	Samuel Robledo	Completada
T-60	Endpoint DELETE /postulaciones/:id	Samuel Robledo	Completada
T-61	Botón de cancelar en vista Mis Postulaciones	Samuel Robledo	Completada
HU-61	Autocompletar correo con carné o apellido	Samuel Robledo	Completada
T-62	Autocompletado de correo institucional en formulario de registro	Samuel Robledo	Completada
HU-59	Ver proyectos destacados	Saul Castillo	Completada
T-63	Endpoint GET /proyectos/destacados	Saul Castillo	Completada
T-64	Sección de proyectos destacados en el dashboard	Saul Castillo	Completada
HU-14	Recuperación de contraseña	Angel Sanabria	Completada
T-65	Endpoints POST /auth/forgot-password y reset-password	Angel Sanabria	Completada
T-66	UI de recuperación de contraseña	Angel Sanabria	Completada

Métricas del Sprint

Gráfica Burndown



Métrica de velocidad



Discusión:

El Sprint 3 puede considerarse exitoso, ya que el equipo logró cerrar la totalidad del trabajo comprometido al final de la iteración. El burndown chart muestra que los puntos pendientes llegaron a cero el 11 de mayo, lo cual indica que las historias y tareas planificadas fueron completadas dentro del periodo establecido. A diferencia de una distribución ideal completamente lineal, el avance real presentó varios descensos marcados en fechas específicas, lo que refleja que el cierre de tareas se concentró en bloques de trabajo y no de manera totalmente uniforme día por día.

Durante los primeros días del sprint se observa una reducción inicial del trabajo pendiente, asociada principalmente a tareas de infraestructura, configuración del entorno productivo y primeras funcionalidades del Sprint 3. Posteriormente, el gráfico mantiene algunos tramos planos, lo que sugiere periodos en los que el equipo continuó trabajando, pero sin cerrar formalmente tareas en Jira. Luego se registran nuevas caídas importantes alrededor del 1 al 3 de mayo y nuevamente entre el 7 y el 11 de mayo, etapa en la que se completaron tareas de integración, validación, corrección de errores, revisión de seguridad y preparación de la entrega final.

La métrica de velocidad también muestra un resultado positivo, porque el trabajo completado coincide con el trabajo comprometido para el Sprint 3. Esto significa que el equipo logró mantener una buena capacidad de entrega y cerrar el alcance definido para la iteración. Además, al comparar con los sprints anteriores, se puede observar una tendencia de mayor estabilidad: el equipo ya venía de completar el alcance de Sprint 1 y Sprint 2, y en esta tercera iteración volvió a cumplir con los puntos planificados, ahora incorporando funcionalidades más maduras y tareas de despliegue en producción.

Sin embargo, el burndown también revela un patrón que debe seguirse mejorando: parte importante del cierre de trabajo se acumuló hacia la segunda mitad del sprint. Esto puede deberse a que varias tareas dependían de otras, como el backend antes del frontend, la configuración de Azure antes de la validación

pública o la implementación de funcionalidades antes de las pruebas finales. Aun así, para próximos sprints sería recomendable registrar avances de forma más continua en Jira, dividir tareas grandes en entregables más pequeños y cerrar subtarefas tan pronto como estén terminadas, para que el gráfico refleje mejor el progreso real del equipo.

En conclusión, el Sprint 3 cumplió con su objetivo principal: fortalecer la plataforma con nuevas funcionalidades y preparar el sistema para un entorno más cercano a producción. El equipo completó el alcance comprometido, mantuvo una velocidad positiva y logró cerrar el sprint con todas las tareas en estado completado. Como oportunidad de mejora, queda continuar reduciendo el patrón de cierre tardío y mejorar la actualización diaria del tablero para tener una visibilidad más precisa del avance durante toda la iteración.

Evidencias de muestras del Incremento a Usuarios



El incremento desarrollado durante el Sprint 3 fue presentado a estudiantes universitarios interesados en participar en proyectos, incluyendo tanto usuarios activos de la plataforma como nuevos interesados en integrarse a proyectos colaborativos. Durante la demostración, se mostró el despliegue de la aplicación en un entorno productivo mediante Azure, así como el flujo completo de las nuevas funcionalidades incorporadas: edición de proyectos, visualización del equipo de trabajo, cancelación de postulaciones, recuperación de contraseña y la sección de proyectos destacados. Asimismo, se destacaron las mejoras en el sistema de notificaciones, que ahora cubre más eventos relevantes dentro del ciclo de vida de los proyectos. Los comentarios recibidos resaltaron positivamente la ampliación de funcionalidades para líderes y estudiantes, así como la estabilidad percibida gracias al despliegue en producción. Sin embargo, se sugirieron mejoras en la consistencia de las notificaciones por correo y en la claridad de algunos mensajes del sistema, especialmente en procesos como la cancelación de postulaciones y la recuperación de contraseña.

Retrospectiva del Sprint

Aspectos positivos

1. **Cumplimiento del alcance planificado del Sprint 3:** El equipo logró completar los 10 bloques principales definidos para esta iteración: hosteo en Azure, tareas transversales, edición de proyectos, notificaciones, equipo del proyecto, cancelación de postulaciones, autocompletado de correo, proyectos destacados y recuperación de contraseña. Esto permitió que el Product Backlog se actualizara con nuevas historias marcadas como completadas en Sprint 3, manteniendo continuidad con los avances de Sprint 1 y Sprint 2.
2. **Avance importante hacia producción:** A diferencia de los sprints anteriores, este sprint no solo agregó funcionalidades visibles para los usuarios, sino que también fortaleció la infraestructura del proyecto. Se trabajó en el despliegue en Azure, configuración del prefijo global /api, Docker Compose, GitHub Actions y documentación del entorno productivo. Esto representa un paso relevante para que la plataforma deje de depender únicamente del entorno local.
3. **Mayor madurez funcional del sistema:** El Sprint 3 completó flujos que hacen que UVG Collab sea más usable para estudiantes y líderes. Entre estos avances destacan la edición de proyectos existentes, la visualización del equipo actual, la cancelación de postulaciones pendientes, la sección de proyectos destacados, la recuperación de contraseña y la mejora del sistema de notificaciones. Estas funcionalidades amplían los flujos base construidos en los dos sprints anteriores.
4. **Mejora parcial en pruebas y validación:** En los sprints anteriores se había identificado como oportunidad de mejora la falta de pruebas automatizadas. En este sprint ya se evidencia un avance, ya que existen pruebas unitarias con Vitest para servicios del backend y pruebas simuladas de flujos en frontend. Aunque todavía no se cuenta con pruebas end-to-end completas, sí hubo una mejora respecto a Sprint 1 y Sprint 2.
5. **Mayor atención a seguridad:** Se revisó el uso de autenticación JWT en los endpoints nuevos y se evidenció que varios endpoints importantes del Sprint 3 están protegidos con JwtAuthGuard, incluyendo edición de proyectos, consulta de equipo, cancelación de postulaciones y notificaciones. Esto muestra una mejor preocupación por controlar el acceso a funcionalidades sensibles.
6. **Continuidad en la división de responsabilidades:** El equipo mantuvo una distribución clara de tareas por integrante. Ángel trabajó principalmente en infraestructura, despliegue y recuperación de contraseña; Saul en equipo del proyecto y proyectos destacados; Vernel en edición de proyecto y pruebas de integración; Derek en notificaciones y ajustes visuales; y Samuel en cancelación de postulaciones, autocompletado y presentación. Esta división permitió trabajar en paralelo sin concentrar todo el incremento en una sola persona.

Aspectos negativos y oportunidades de mejora

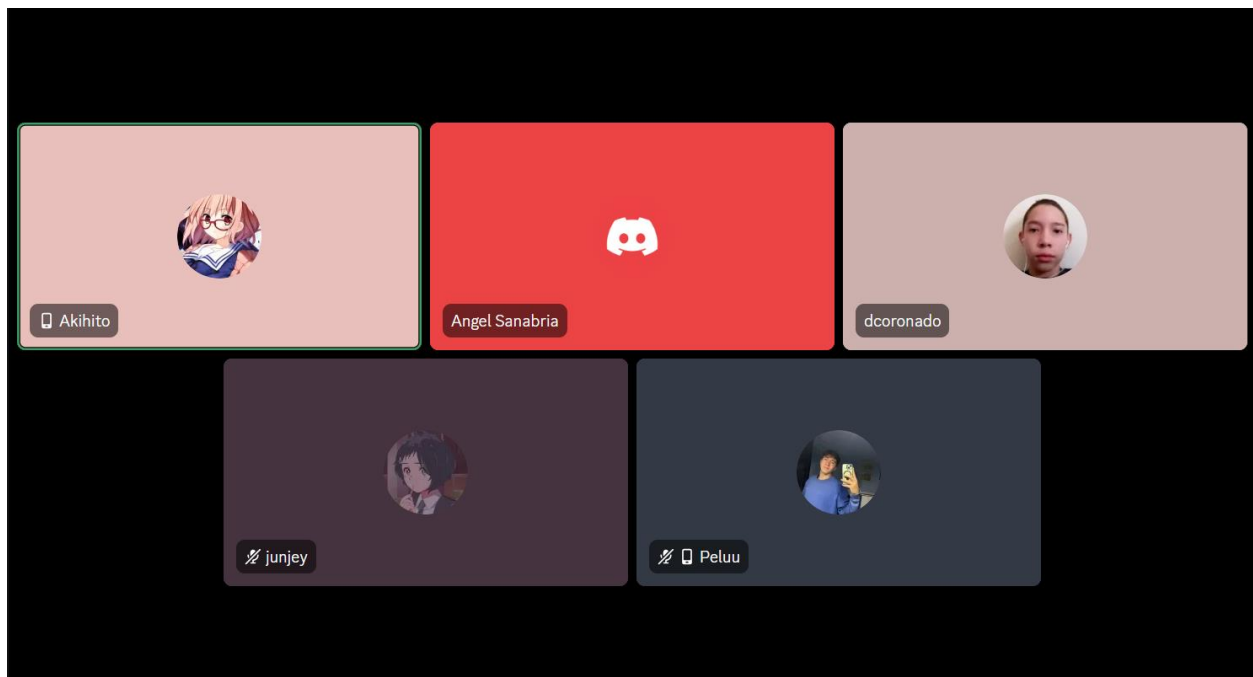
1. **Persistencia del patrón de cierre tardío en el burndown:** Aunque el sprint cerró con todas las tareas completadas, el burndown muestra nuevamente un avance no uniforme. Hubo tramos planos donde el trabajo seguía en desarrollo, pero no se cerraban tareas formalmente en Jira. Esto repite un problema identificado desde Sprint 1 y Sprint 2: el equipo trabaja, pero no siempre actualiza el tablero de forma incremental.
2. **Infraestructura parcialmente evidenciada:** Aunque el despliegue en Azure está documentado y existe evidencia del pipeline de GitHub Actions, no hay un archivo `nginx.conf` versionado en el repositorio. Limitando la trazabilidad técnica de la configuración exacta del reverse proxy y obliga a depender de evidencia externa del servidor.
3. **Pipeline de producción con oportunidad de mejora:** El pipeline de GitHub Actions está evidenciado, sin embargo, no ejecuta explícitamente `prisma migrate deploy`. Esto es un riesgo para producción, porque para futuras migraciones podrían requerir ejecución manual si no se automatiza correctamente dentro del flujo CI/CD.
4. **Pruebas automatizadas todavía incompletas:** No existen pruebas end-to-end reales y que algunos métodos, como la cancelación de postulaciones, no cuentan con prueba unitaria específica. Por tanto, la deuda técnica de pruebas se redujo, pero no quedó completamente resuelta.
5. **Bugs residuales en frontend:** Todavía hay errores menores en la UI, como el uso de un endpoint incorrecto en la página de notificaciones para marcar todas como leídas y la falta de importación de `apiFetch` en la sección de proyectos destacados.
6. **Funcionalidad de recuperación de contraseña parcialmente completa:** El flujo de recuperación de contraseña fue implementado con endpoints y UI, pero el envío real de correo todavía no está completamente empleado. La historia depende de que el usuario pueda recibir efectivamente el enlace de recuperación.

Medidas a tomar para el Sprint 4

1. **Actualizar Jira de forma diaria y cerrar tareas de manera incremental:** Cada integrante debe mover sus tareas a “In Progress” al comenzar y a “Done” cuando estén funcionalmente terminadas. Esto permitirá que el burndown refleje mejor el avance real y evitará que el cierre se concentre al final del sprint.
2. **Versionar la configuración de infraestructura crítica:** La configuración de Nginx, o al menos una plantilla de referencia, debería quedar documentada dentro del repositorio. Esto reduciría la dependencia de configuraciones manuales en la VM de Azure.

3. **Mejorar el pipeline CI/CD:** Se recomienda agregar explícitamente prisma migrate deploy al flujo de despliegue, junto con validaciones claras para asegurar que las migraciones de base de datos se apliquen correctamente en producción.
4. **Corregir bugs residuales antes de iniciar nuevas historias:** Antes de ampliar el alcance del Sprint 4, se deberían corregir los problemas detectados en notificaciones y proyectos destacados, especialmente el endpoint incorrecto para marcar todas como leídas y la importación faltante de apiFetch.
5. **Completar el flujo real de correo para recuperación de contraseña:** El Sprint 4 debe cerrar la deuda pendiente del envío real de correos, configurando correctamente el servicio de email y probando el flujo completo desde solicitud hasta restablecimiento de contraseña.
6. **Incorporar pruebas end-to-end básicas:** El equipo ha avanzado con pruebas unitarias y simuladas, pero el siguiente paso debería ser incluir al menos pruebas E2E para los flujos críticos: login, postulación, cancelación de postulación, edición de proyecto y recuperación de contraseña.
7. **Mantener tareas más pequeñas y verificables:** Para evitar cierres tardíos, las historias grandes deben dividirse en tareas pequeñas que puedan completarse y validarse en uno o dos días. Esto también permitirá detectar errores antes de la integración final.

Evidencias de Reuniones



Durante el desarrollo del Sprint 3, el equipo mantuvo reuniones grupales semanales a través de Discord, en las cuales cada integrante reportaba el avance de sus tareas asignadas, compartía impedimentos encontrados y coordinaba dependencias entre subtareas de backend y frontend. Estas sesiones permitieron dar

seguimiento continuo al estado de las historias de usuario, resolver dudas técnicas de forma colaborativa y ajustar prioridades cuando fue necesario. La comunicación constante contribuyó a que el equipo alcanzara el 100% de completitud de las historias planificadas para el sprint.

Conclusiones

- El Sprint 3 representó una etapa importante de consolidación para UVG Collab, ya que permitió avanzar más allá de las funcionalidades base construidas en los sprints anteriores. La incorporación del hosting en Azure, la configuración del entorno productivo, el uso de Docker, el pipeline de despliegue y la documentación técnica demuestran que el equipo comenzó a preparar la plataforma para operar en un ambiente más cercano a producción.
- Desde el punto de vista funcional, el sprint amplió significativamente el alcance del sistema. Funcionalidades como la edición de proyectos, la visualización del equipo actual, la cancelación de postulaciones, los proyectos destacados, el autocompletado del correo institucional, las notificaciones extendidas y la recuperación de contraseña hacen que la plataforma sea más completa y útil para estudiantes y líderes.
- El cumplimiento de todas las tareas planificadas evidencia que el equipo mantuvo una buena capacidad de entrega durante la iteración. Además, la división de responsabilidades permitió que cada integrante aportara desde un área específica del sistema, facilitando el avance paralelo entre infraestructura, backend, frontend, pruebas y documentación.
- A pesar del cumplimiento del alcance, el burndown chart muestra que todavía existe un patrón de cierre tardío. Esto indica que el equipo debe mejorar la actualización constante de Jira, registrar avances de forma más ordenada y cerrar subtareas en cuanto estén funcionalmente terminadas, para reflejar mejor el progreso real del sprint.
- También se identifican oportunidades de mejora en calidad técnica y sostenibilidad. Aunque hubo avances en pruebas y validación, todavía es necesario fortalecer la cobertura con pruebas end-to-end, corregir errores residuales en la interfaz, completar el flujo real de envío de correos para recuperación de contraseña y mejorar la documentación de configuraciones externas como Nginx.
- El Sprint 3 puede considerarse exitoso porque cumplió con el alcance comprometido, fortaleció la plataforma con nuevas funcionalidades y permitió avanzar hacia un entorno productivo. Para el Sprint 4, el principal reto será mantener la velocidad de entrega alcanzada, pero con un proceso más ordenado, verificable y sostenible.